

LIST MANIPULATION

Python Mini Project Report

Designed and maintained by
Arham Pathan — 26BAI10339

Project Type: Menu-driven Python application

Topic: Python Lists and List Manipulation

Prepared from the uploaded project document and its embedded implementation/output screenshots.

1. Abstract

List Manipulation is a menu-driven Python program designed to help a user manage the elements of a list. The project provides operations for entering, displaying, searching, adding, updating, deleting, sorting, and finding the maximum/minimum values of a list. A continuous main menu lets the user perform multiple operations before choosing Exit.

2. Problem Statement

The objective is to create an interactive Python program that provides common list-management operations through a simple menu. The user should be able to modify and inspect the list without restarting the program after every operation.

3. Objectives

- Understand the Python list data structure.
- Implement common list methods in a practical application.
- Use a menu-driven interface with while loops and conditional statements.
- Perform searching, insertion, updating, deletion and sorting.
- Display maximum and minimum values from the list.
- Provide a clear exit path and invalid-choice handling.

4. Functional Specification

Option	Functionality	Python operation(s)
1	Enter list	Input / list assignment
2	Display list	Print
3	Search an element	in, index()
4	Add	append(), extend(), insert()
5	Update element	Index assignment
6	Delete	pop(), remove(), clear()
7	Sort list	sort(), sort(reverse=True)
8	Maximum / Minimum	max(), min()
9	Exit	break

5. Algorithm / Program Flow

Step 1: Start the program and initialize an empty list.

Step 2: Display the main menu with options 1–9.

Step 3: Read the user's choice.

Step 4: Execute the operation associated with the selected choice.

Step 5: For Add/Delete/Sort, display the relevant submenu and perform the selected list operation.

Step 6: Return to the main menu after the operation.

Step 7: If option 9 is selected, display a thank-you message and terminate the loop.

Step 8: For an invalid main-menu choice, display an invalid-choice message and continue.

6. Pseudocode

```
START
Create empty list A

REPEAT
    Display main menu
    Read CHOICE

    IF CHOICE = 1
        Read a list into A
    ELSE IF CHOICE = 2
        Display A
    ELSE IF CHOICE = 3
        Read element X
        IF X exists in A
            Display index of X
        ELSE
            Display not-found message
    ELSE IF CHOICE = 4
        Display Add submenu
        Perform append / extend / insert
    ELSE IF CHOICE = 5
        Read index and replacement element
        Update A[index] when index is valid
    ELSE IF CHOICE = 6
        Display Delete submenu
        Perform pop / remove / clear
    ELSE IF CHOICE = 7
        Display Sort submenu
        Perform ascending / descending sort
    ELSE IF CHOICE = 8
        Display max(A) and min(A)
    ELSE IF CHOICE = 9
        Display thank-you message
        EXIT LOOP
    ELSE
        Display invalid-choice message
UNTIL EXIT

END
```

7. Implementation Details

The main menu shown on page 2 of the source document contains nine options: Enter list, Display list, Search an element, Add, Update element, Delete, Sort list, Display max/min value, and Exit.

The search section on page 3 checks whether an element exists with **if c in a** and then reports its index using **a.index(c)**.

The Add submenu on page 4 demonstrates **append()**, **extend()**, and **insert()** for adding one or multiple elements.

The Update operation on page 5 asks for an index and replacement element and updates the list when the index is within the list length.

The Delete submenu on page 6 provides deletion by index with **pop()**, deletion by value with **remove()**, and clearing with **clear()**.

The Sort submenu on page 7 provides ascending sorting with **sort()** and descending sorting with **sort(reverse=True)**.

Page 8 implements maximum/minimum using **max(a)** and **min(a)**, followed by the Exit option and invalid-choice handling.

8. Test Cases / Observed Operations

Test	Operation	Expected/Observed Result
TC-01	Enter list	List is stored and displayed.
TC-02	Display list	Current list is printed.

Test	Operation	Expected/Observed Result
TC-03	Search existing element	Element index is displayed.
TC-04	Append / Extend / Insert	List is modified and printed.
TC-05	Update valid index	Selected element is replaced.
TC-06	Delete by index/value	Selected element is removed.
TC-07	Clear list	List becomes empty.
TC-08	Ascending / descending sort	List is reordered.
TC-09	Max / min	Maximum and minimum are displayed.
TC-10	Exit	Program prints a thank-you message and terminates.

9. Basic Complexity Notes

For a list of size n , membership/search using `in` and `index()` can take $O(n)$ time in the general case. Appending an item is typically $O(1)$ amortized, while inserting/removing at an arbitrary position can require $O(n)$ shifting. Python's built-in list sorting is typically $O(n \log n)$.

10. Limitations and Possible Improvements

- The supplied implementation uses `eval(input(...))` for Python-style input. For a production program, safer parsing and explicit type conversion are recommended.
- Additional validation can be added for non-numeric lists before `max()`, `min()`, and sorting operations.
- Update and delete operations can provide clearer messages when an index or value is invalid.
- Separate functions can be created for each menu operation to improve maintainability.
- A GUI or file-based save/load feature could be added as a future extension.

11. Conclusion

The List Manipulation project demonstrates practical use of Python lists through an interactive menu. It covers the major list operations required in the supplied project and provides console examples for the implemented features. The project is suitable for demonstrating list methods, loops, conditional statements, and user input.

Appendix A — Source Project Screenshots

The following images are extracted from the uploaded project document and preserve the original code/output evidence.

Source screenshot 1

```
"IDE Shell 3.10.7"
File Edit Shell Debug Options Window Help

-----MAIN MENU-----

** 1-Enter list **
** 2-Display list **
** 3-Search an element **
** 4-Add **
** 5-Update element **
** 6-Delete **
** 7-Sort list **
** 8-Display max/min value **
** 9-Exit **

Enter a choice :1
Enter a list :[2,4,6,8,10]

You entered the list - [2, 4, 6, 8, 10]

-----MAIN MENU-----

** 1-Enter list **
** 2-Display list **
** 3-Search an element **
** 4-Add **
** 5-Update element **
** 6-Delete **
** 7-Sort list **
** 8-Display max/min value **
** 9-Exit **

Enter a choice :2
The list is :
[2, 4, 6, 8, 10]
```

Source screenshot 2

```
"IDE Shell 3.10.7"
File Edit Shell Debug Options Window Help

** 5-Update element **
** 6-Delete **
** 7-Sort list **
** 8-Display max/min value **
** 9-Exit **

Enter a choice :3
Enter an element to search :6
The element is at the index - 5
Press any key to continue...

-----MAIN MENU-----

** 1-Enter list **
** 2-Display list **
** 3-Search an element **
** 4-Add **
** 5-Update element **
** 6-Delete **
** 7-Sort list **
** 8-Display max/min value **
** 9-Exit **
```

Source screenshot 3

```
"IDE Shell 3.10.7"
File Edit Shell Debug Options Window Help

** 9-Exit **

Enter a choice :4

1 - To add an element at the end
2 - To add mutiple elements at the end
3 - To add elements in between the list
4 - Exit

Enter your choice :1
Enter the element to add at the end of the list :5

The list is -
[1, 2, 3, 4, 5, 6, 7, 5]

Press any key to continue...

1 - To add an element at the end
2 - To add mutiple elements at the end
3 - To add elements in between the list
4 - Exit
```

Source screenshot 4

```

IDLE Shell 3.10.7
File Edit Shell Debug Options Window Help

Enter a choice :5
Enter the index at which you want to update the element -2
Enter the element to replace :8
The list is -
[1, 2, 8, 4, 5, 6, 7, 5]

** 7-Sort list **
** 8-Display max/min value **
** 9-Exit **

----MAIN MENU----

** 1-Enter list **
** 2-Display list **
** 3-Search an element **
** 4-Add **
** 5-Update element **
** 6-Delete **
** 7-Sort list **
** 8-Display max/min value **
** 9-Exit **
```

Source screenshot 5

```

IDLE Shell 3.10.7
File Edit Shell Debug Options Window Help

Enter a choice :6

1 - To delete an element by index
2 - To delete an element by value
3 - To clear the list
4 - Exit

Enter your choice :1
Enter the index - 2

The list is -
[1, 2, 4, 5, 6, 7, 5]

Press any key to continue...

1 - To delete an element by index
2 - To delete an element by value
3 - To clear the list
4 - Exit

** 5-Update element **
** 6-Delete **
** 7-Sort list **
** 8-Display max/min value **
** 9-Exit **
```

Source screenshot 6

```

IDLE Shell 3.10.7
File Edit Shell Debug Options Window Help

Enter a choice :7

1 - To sort the list in ascending order
2 - To sort the list in descending order
3-Exit

Enter your choice:1
Ascending order : [1, 2, 4, 5, 5, 6, 7]

1 - To sort the list in ascending order
2 - To sort the list in descending order
3-Exit

** 2-Display list **
** 3-Search an element **
** 4-Add **
** 5-Update element **
** 6-Delete **
** 7-Sort list **
** 8-Display max/min value **
** 9-Exit **
```

Source screenshot 7

```

IDLE Shell 3.10.7
File Edit Shell Debug Options Window Help

Enter a choice :8

Maximum value : 7
Minimum value : 1

** 7-Sort list **
** 8-Display max/min value **
** 9-Exit **
```

Source screenshot 8

```
-----MAIN MENU-----  
  
** 1-Enter list **  
** 2-Display list **  
** 3-Search an element **  
** 4-Add **  
** 5-Update element **  
** 6-Delete **  
** 7-Sort list **  
** 8-Display max/min value **  
** 9-Exit **  
  
Enter a choice :9  
Thank you for using this python program  
>>>
```

